



TEORÍA DE ALGORITMOS
CURSO ECHEVARRIA

Trabajo Práctico 1

May 9, 2025

- Juan Claros - 111254
- Yoel Gaston Arlia - 111520
- Guido Cuppari - 111172
- Josue Daniel Mamani - 111514

Índice

1	Problema 1	3
1.1	Supuestos	3
1.2	Diseño	3
1.3	Pseudocodigo	4
1.4	Análisis de Complejidad	4
1.5	Seguimiento	4
1.6	Casos de prueba y mediciones	5
1.7	Resultados	5
1.8	Generacion de sets	5
2	Problema 2	6
2.1	Supuestos	6
2.2	Diseño	6
2.3	Análisis de complejidad	6
2.4	Seguimiento de Ejemplo	6
2.5	Casos de prueba y mediciones	7
2.6	Resultados	7
2.7	Referencias	7
2.8	Generacion de sets	7
3	Problema 3	8
3.1	Enunciado y supuestos	8
3.2	Diseño	8
3.3	Pseudocodigo	8
3.4	Seguimiento	9
3.5	Complejidad	10
3.6	Set de datos	10
3.7	Tiempos de ejecución	10
3.8	Informe de resultados	11

1 Problema 1

1.1 Supuestos

- Se supone que las cargas estan almacenadas en una lista Q , de manera equidistante y cada posicion de la lista representa la posicion de las particulas

1.2 Diseño

- En el main, se generan los archivos que almacenan las cargas, donde cada posición tiene un valor de carga distinta, desde -10 hasta 10.
- La función `calcular_fuerza` aplica un algoritmo de División y Conquista, dividiendo el problema en subproblemas y juntando los resultados. El arreglo de cargas Q se divide recursivamente en mitades hasta llegar a subconjuntos de tamaño 1. Después de calcular las fuerzas dentro de cada mitad, se computan las interacciones cruzadas entre las partículas de la mitad izquierda y la mitad derecha. Esto asegura que todas las combinaciones (i, j) con $i < j$ sean consideradas exactamente una vez. Las fuerzas se acumulan: $F[i]$ recibe una fuerza negativa (j lo repele o atrae) y $F[j]$ una positiva (i lo repele o atrae).
- Se utilizan dos estructuras de datos principales: las listas Q y F . La lista Q contiene los valores enteros de las cargas, y permite acceder en tiempo constante a cualquier elemento, lo cual es útil para recorrer y multiplicar cargas en distintos rangos. La lista F tiene el mismo tamaño y almacena las fuerzas acumuladas como números flotantes. Ambas listas permiten acceso aleatorio y modificación eficiente, lo cual es fundamental para implementar el algoritmo de División y Conquista sin necesidad de estructuras adicionales. Estas estructuras nativas de Python resultan adecuadas por su simplicidad y eficiencia.

1.3 Pseudocódigo

```

1 def calcular_fuerza(q, C=1):
2     n = len(q)
3     F = [0.0] * n
4     t1 = time.time()
5     def divide_conquista(inicio, fin):
6
7         if fin - inicio <= 1:
8             return
9         medio = (inicio + fin) // 2
10        # Resolver izquierda y derecha por separado
11        divide_conquista(inicio, medio)
12        divide_conquista(medio, fin)
13
14        # Calcular las interacciones cruzadas entre mitades
15        for i in range(inicio, medio):
16            for j in range(medio, fin):
17                distancia = j - i
18                fuerza = C * q[i] * q[j] / (distancia *
19                distancia)
20                F[j] += fuerza      # i < j -> fuerza positiva
21                F[i] -= fuerza      # j > i -> fuerza negativa
22        divide_conquista(0, n)
23        t2 = time.time()
24        print(F)
25        print("Tiempo de ejecuci n:", t2 - t1)
26        return F
27

```

1.4 Analisis de Complejidad

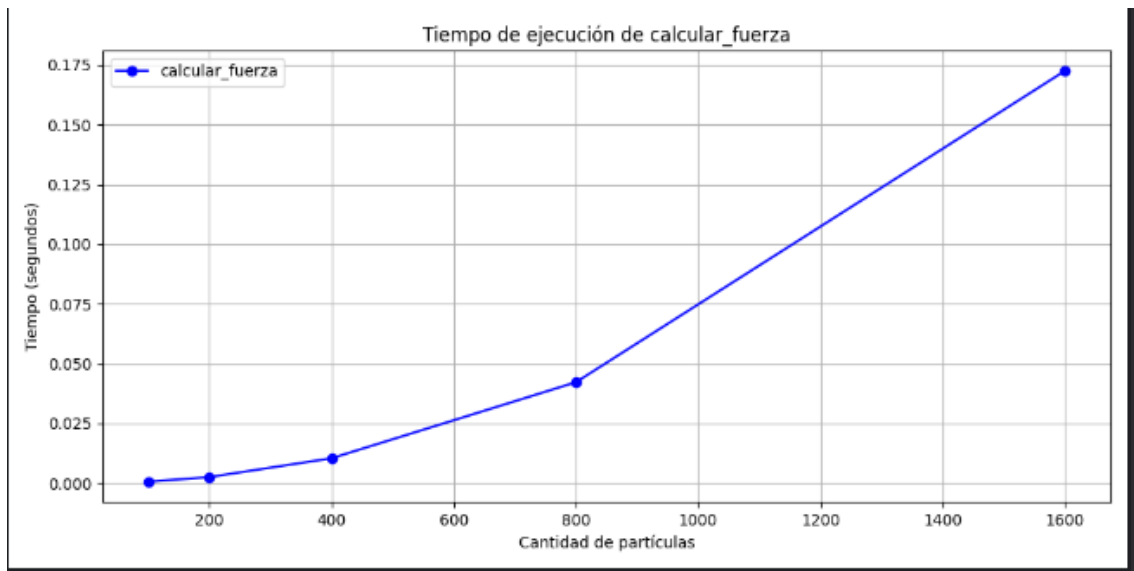
- La declaracion de variables es $O(1)$, la division en subproblemas recursivamente es $O(\log n)$, luego con los dos for anidados se trata de, en el peor de los casos, $O(N^2)$.

1.5 Seguimiento

- Se toma como ejemplo el vector de cargas $q = [2, -1, 3, 1]$. El algoritmo comienza inicializando un arreglo de fuerzas $F = [0.0, 0.0, 0.0, 0.0]$. Luego, se aplica recursivamente la estrategia de división y conquista.
- En el primer nivel de recursión, el vector se divide en dos mitades: izquierda $[2, -1]$ (índices 0–1) y derecha $[3, 1]$ (índices 2–3). Se continúa dividiendo hasta llegar a subarreglos de tamaño 1, sobre los cuales no se realiza ninguna operación.
- En la etapa de combinación, se calculan las interacciones cruzadas entre las mitades:

- Para $i = 0$ (carga 2) y $j = 2$ (carga 3): distancia $d = 2$, fuerza $F = \frac{2 \cdot 3}{2^2} = 1.5$. Se actualiza: $F[0] = -1.5$, $F[2] = 1.5$.
 - Para $i = 0$ (carga 2) y $j = 3$ (carga 1): $d = 3$, $F = \frac{2 \cdot 1}{3^2} \approx 0.222$. Se actualiza: $F[0] = -1.722$, $F[3] = 0.222$.
 - Para $i = 1$ (carga -1) y $j = 2$ (carga 3): $d = 1$, $F = \frac{-1 \cdot 3}{1^2} = -3.0$. Se actualiza: $F[1] = 3.0$, $F[2] = -1.5$.
 - Para $i = 1$ (carga -1) y $j = 3$ (carga 1): $d = 2$, $F = \frac{-1 \cdot 1}{2^2} = -0.25$. Se actualiza: $F[1] = 3.25$, $F[3] = -0.028$.
- Al finalizar todas las interacciones cruzadas, el vector de fuerzas contiene los valores netos acumulados para cada partícula: $F = [-1.722, 3.25, -1.5, -0.028]$, lo que representa la fuerza total que cada carga recibe del sistema.

1.6 Casos de prueba y mediciones



1.7 Resultados

- Los resultados, según la cantidad de partículas con las cantidades de 100, 200, 400, 600, 800 y 1000, son los esperados y coinciden con la complejidad.

1.8 Generación de sets

- Los sets son realizados a partir de una función randomizada, que generan archivos de texto con listas que representan las cargas a probar.

2 Problema 2

2.1 Supuestos

- Se supone que las distancias se encuentran en una lista, donde cada posición es la distancia a la anterior posición, y cada posición representa una parada. El resultado se genera almacenando los subíndices de cada parada.

2.2 Diseño

- En el main, se generan los archivos que almacenan las distancias a evaluar con una semilla fija, luego se generan los archivos donde van los resultados y se llama a la función que realiza el cálculo.
- La función `generar_datos`, genera una lista con datos aleatorios basándose en la semilla recibida, estos datos son números enteros de 0 a 7 inclusive y se genera una lista de largo recibido que es retornada.
- La función `calcular_mejor_distancia` recorre mientras que el elemento que estoy evaluando es menor que el largo de la lista de paradas, y luego avanza mientras que la suma de distancias sea menor que 7, modificando la posición actual y luego una vez que la suma exceda 7 se agrega a la lista de resultados el índice -1 que sería la posición que cumple el rango, luego continúa siguiendo esta misma idea, y agrega el último paraje para llegar hasta el final, imprime el tiempo que tardó y devuelve una lista con los resultados.

2.3 Analisis de complejidad

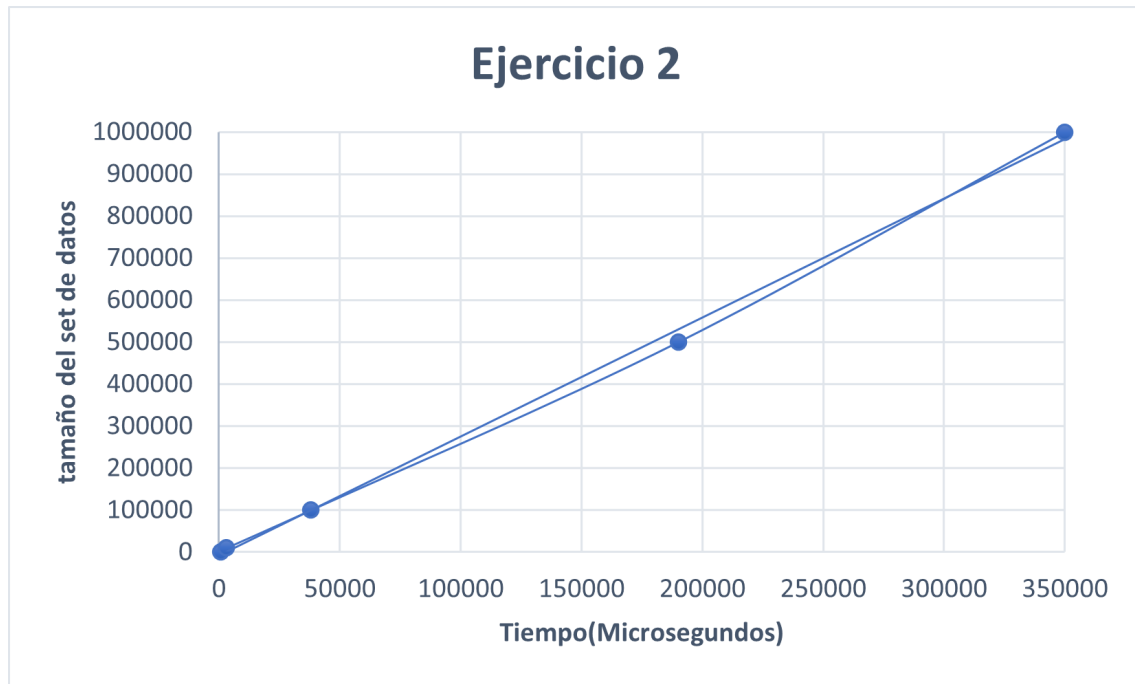
- La parte de generar los datos no la tengo en cuenta en el análisis, solo la parte de calcular los resultados, `time` es $O(1)$, inicializar variables también es $O(1)$, `len(distancias)`, es $O(1)$, imprimir y utilizar `append` es $O(1)$, los `while` anidados, comparten la condición de que la posición sea menor que el largo y en cada iteración se avanza al menos una vez, por lo que la complejidad de esta parte es $O(n)$, como conclusión, el resultado es $O(n)$.

2.4 Seguimiento de Ejemplo

- Vamos a hacer un ejemplo con la lista de distancias = [4, 3, 6, 6, 1, 7], Al llegar a la función se inicializan las variables, y el largo queda en 5, se entra al primer `while` ya que `i` está en 0 que es menor que el largo, luego se entra al otro `while`, repitiendo la condición y evaluando que el elemento en la lista en la posición `i` (0) sea menor o igual que 7, como es el caso (4), se entra al `while` y se suma el actual en `auxiliar`(4) y se aumenta la posición(1), como ahora la posición es 1 y la suma en `auxiliar` es 4, evaluando la condición en `auxiliar` + el valor en la nueva posición(1) (valor = 3), como `auxiliar`(7) es menor o igual que 7, se suma este valor parcial en `auxiliar`(7) y se suma en 1 la posición(2), ahora al evaluar la condición de menor o igual que 7 va a dar `false` porque el valor es 13 que es mayor, se sale de este `while`, se guarda el índice de la

posicion actual a la anterior (2-1), y se continua iterando de esta forma hasta llegar a la ultima posicion (5), donde esta posicion ya no es menor que el largo por lo que se deja de iterar y se agrega este indice, se calcula e imprime el tiempo final. Devuelve una lista con los resultados obtenidos.

2.5 Casos de prueba y mediciones



2.6 Resultados

- Los resultados para 1000, 10000, 100000, 500000 y 1000000, son lo esperado y coincide con la complejidad analizada $O(n)$.

2.7 Referencias

- Van Rossum, G., & Python Software Foundation. (2023). Built-in Functions: open. Retrieved from <https://docs.python.org/3/library/functions.html#open>
- Sedgewick, R., & Wayne, K. (2011). Algorithms (4th ed.). Addison-Wesley.

2.8 Generacion de sets

- para generar los sets se realiza de la siguiente manera, utilizando 1 como semilla para el largo 1000, 2 como semilla para el largo 100000, 3, como semilla para largo 100000, 4 como semilla para largo 500000, y 5 como semilla para largo 1000000.

```

1 def generar_datos(largo, semilla):
2     random.seed(semilla)
3     return [random.randint(0, MAXIMA_DISTANCIA) for _ in range(
    largo)]

```

3 Problema 3

3.1 Enunciado y supuestos

Una suma encadenada es una secuencia de números $a_0, a_1, \dots, a_n$ tal que $a_0=1$ y para cada $k \geq 0$: $a_k = a_i + a_j$ para algún i, j, k . Por ejemplo, 1, 2, 3, 6 es una suma encadenada para 6 de longitud 3 ya que:

- $2=1+1$
- $3=2+1$
- $6=3+3$

Para desarrollar un algoritmo de Backtracking que pueda calcular una suma encadenada de longitud mínima para un entero positivo n y también tiene que ser $n \geq 2$ ya que no tiene sentido calcular un $n = 1$ ya que es sencillo.

3.2 Diseño

Para este ejercicio, se utilizó backtracking el cual se implementó utilizando la recursividad, se pensó en un árbol que contenga las posibles soluciones para un n buscado siendo la raíz 1 y siendo los hijos las posibles sumas encadenadas posibles por las sumas de los nodos recorridos desde la raíz hasta un hijo dado, el recorrido de la raíz hasta el hijo sería la longitud de la suma encadenada. La manera de recorrer el árbol es meramente parecida a DFS. por ejemplo tenemos como recorrido inicial a recorrido = [1] las posibles sumas encadenadas posibles son $2 = 1 + 1$ y luego 2 tiene dos posibles hijos que serían $4 = 2 + 2$ y $3 = 3 + 1$ pero el algoritmo va recorriendo el árbol de derecha a izquierda y así iterando suma por suma terminamos recorriendo todo;

3.3 Pseudocódigo

```

1  backtracking(camino, objetivo, inicio, resultado, suma)
2      #caso base
3      si suma == objetivo
4          si len(resultado) == 1 and len(resultado[0]) > len(
    recorrido)
5              resultado = recorrido
6              si len(resultado) == 0:
7                  resultado = recorrido
8              return
9      #corte o poda

```

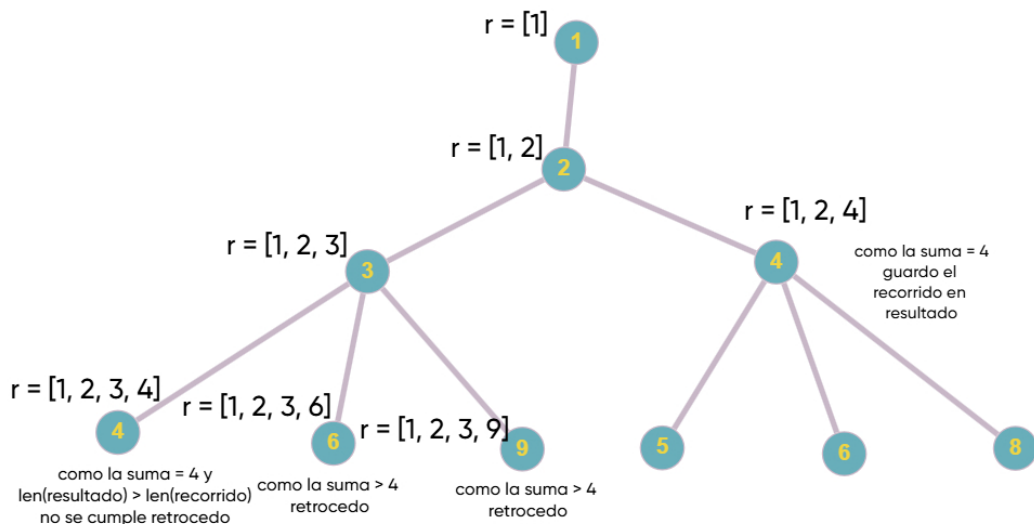


```

10     si suma > n or len(resultado) != 0 and len(recorrido) > len(
resultado[0]):
11         return
12     indice = len(recorrido) - 1
13     #recorriendo las posibles soluciones
14     siempre que len(recorrido) >= 1 and indice <= len(recorrido)
- 1 and indice >= 0
15     longitud = len(recorrido) - 1
16     si es la primera iteracion entonces
17         suma = recorrido[indice] + recorrido[longitud]
18         a recorrido agregar suma
19         backtracking(recorrido,n,len(recorrido) - 1,resultado,
suma)
20     return resultado
21     sino
22         suma = recorrido[indice] + recorrido[longitud]
23         a recorrido agregar suma
24         si indice > 0
25             backtracking(recorrido,n,len(recorrido) - 1,resultado,
suma)
26     else:
27         backtracking(recorrido,n,indice,resultado,suma)
28         eliminar el ultimo elemento de rrecorrido
29         indice = indice - 1
30
31

```

3.4 Seguimiento



Para el seguimiento tomamos como $n = 4$ para encontrar la menor suma encaenada dando como resultado una longitud de 2 los cuales serian $2 = 1 + 1$ y $4 = 2 + 2$, que se vera mas claro como seria el seguimiento para encontrar en la imagen siguiente.

3.5 Complejidad

El algoritmo que implementa backtracking explora todas las posibles sumas encadenadas que se pueden hacer con los elementos de un estado de recorrido, por ejemplo si $\text{recorrido} = [1, 2, 4]$ las posibles sumas encadenadas son

- $5 = 1 + 4$
- $6 = 4 + 2$
- $8 = 4 + 4$

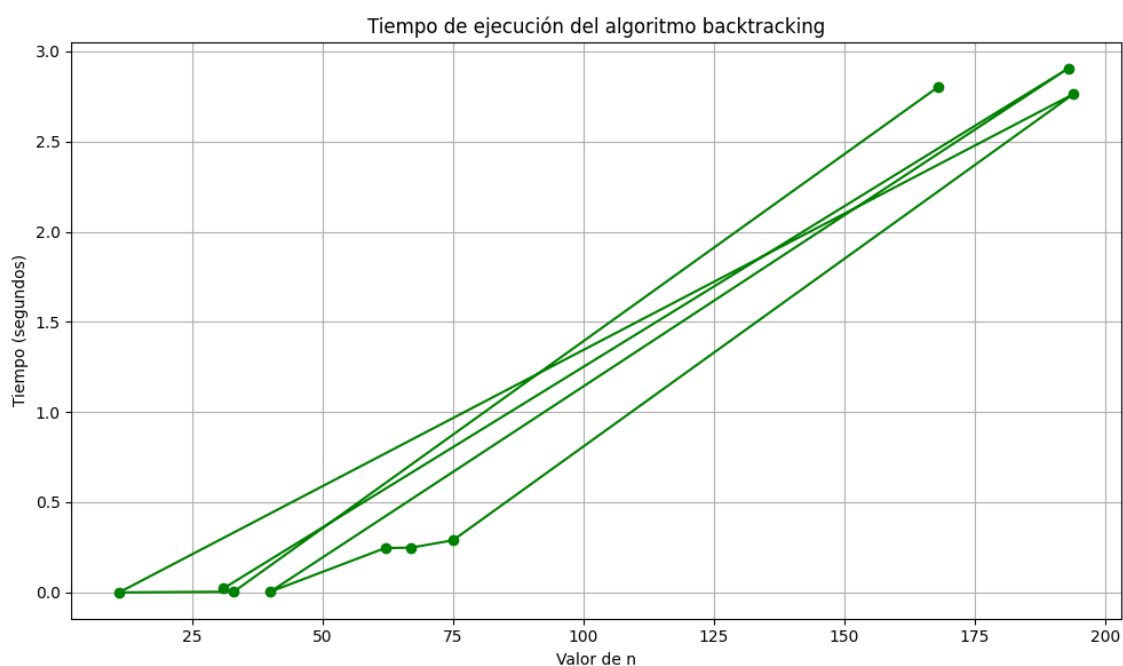
cada una de estas combinaciones posibles genera una nueva llamada recursiva, por lo tanto cada llamada recursiva puede tener múltiples hijos el cual va en aumento si el n es mayor y a medida que la variable recorrido va en aumento también aumentan los hijos generados de manera exponencial en el peor de los casos, por lo tanto la complejidad en el peor de los casos es $O(2^n)$

3.6 Set de datos

los sets de datos para el problema se generan de con el siguiente código

```
1 def generar_sets(cantidad, semilla=42):  
2     random.seed(semilla)  
3     sets = []  
4     for _ in range(cantidad):  
5         n = random.randint(5, 200)  
6         sets.append(n)  
7     return sets  
8
```

3.7 Tiempos de ejecución



3.8 Informe de resultados

Por medio del grafico corresponde como $O(2^n)$ en el peor de los casos que seria que se hagan todas las posibles sumas encadenas, en le grafico vemos que al pricipio los valores de n como por ejemplo entre $n = 0$ hasta $n = 50$ el tiempo es casi 0 pero despues con valor aprox 150 se eleva exponencialmente y los demas valores, el cual es un compotamiento de la complejidad exponencial, tambien pueden haber valores de n que podriamos pensar que su tiempo aumentaria exponencialmente pero tambien puede que tarde menos de los pensado ya que el algoritmo puede tambien podar y eso causaria que tarde menos y eso reduciria su tiempo.